

Designing Domain-Specific Conversational AI for IT Support: A Practical Approach Using LangGraph, LangChain, and Fine-Tuned LLMs

Author: Deep Patel

Master's in CS: Artificial Intelligence and Machine Learning, Woolf University

Abstract- The increasing use of AI in business settings has created new opportunities for intelligent automation, especially in IT support and customer service. This research presents a practical framework for designing and deploying industry-specific conversational AI systems tailored for the Technology and IT sector. By fine-tuning pre-trained Large Language Models (LLMs) sourced from Hugging Face and integrating them with modular orchestration frameworks such as LangGraph and LangChain, the study demonstrates how a scalable, responsive, and context-aware chatbot can be built using real-world data sources like technical documentation, FAQs, and client-specific web content.

The chatbot is designed to enhance IT support workflows by understanding nuanced technical queries, reducing human workload, and improving response times. The research details the end-to-end lifecycle: from data acquisition and model fine-tuning to deployment and evaluation. Results indicate strong performance in delivering accurate, relevant, and domain-specific answers across diverse IT scenarios.

This paper not only bridges the gap between general-purpose LLMs and enterprise needs but also provides a reusable blueprint for IT firms aiming to offer customised AI-powered solutions to their clients. The findings contribute to the broader discourse on applied NLP and conversational AI in real-world enterprise settings.

Index Terms- Conversational AI, Tech Support, LangChain, LangGraph, Large Language Models, Hugging Face, NLP, Custom Chatbot, Industry-Specific AI, IT Services

I. INTRODUCTION

I.1 Background and Motivation

In the past decade, the rapid progression of Artificial Intelligence (AI) and Natural Language Processing (NLP) has drastically reshaped human-computer interaction. One of the most promising applications of AI lies in conversational systems, which use Large Language Models (LLMs) to simulate human-like dialogue. These AI-driven interfaces are not just limited to general queries and customer service—they are now being designed for deeply technical domains, such as IT support, cybersecurity, software development, and cloud infrastructure management.

The Technology and Information Technology (IT) industry operates within a complex, ever-evolving landscape where

precision, uptime, and technical accuracy are paramount. Whether it's a software crash, a server configuration issue, or cloud infrastructure latency, immediate resolution is critical. Traditionally, such issues were handled by human support teams through hierarchical ticketing systems. While effective, these systems often face limitations like high turnaround time, language barriers, inconsistent documentation, and a lack of contextual memory.

With the advent of LLMs such as GPT-3, Falcon, and Mistral, which are trained on extensive and diverse corpora, there is now an opportunity to fine-tune these models to understand domain-specific terminology, workflows, and support interactions. However, while general-purpose LLMs offer fluent and grammatically correct responses, they often fail to deliver the domain-specific accuracy and technical rigor required for enterprise IT support. This necessitates a paradigm shift—from using "off-the-shelf" LLMs to custom fine-tuned models enriched with industry-relevant data.

I.2 Research Problem

Many IT companies face a common challenge: how to automate and scale technical support without compromising the accuracy, personalization, or reliability of responses. Existing chatbots often use rule-based logic, lack contextual awareness, and are unable to integrate data from multiple sources such as user manuals, API documentation, troubleshooting guides, and system logs. These shortcomings result in poor user experience, higher operational costs, and reliance on human escalation for even moderately complex issues.

To address this challenge, our research focuses on the development of a domain-specific conversational AI agent that can deliver technical support within the IT industry. This agent is not generic; it is trained on custom data—PDF documents, web-based technical documentation, system FAQs, and knowledge base articles—and orchestrated using modern frameworks like LangGraph and LangChain, allowing structured, stateful interactions with users.

I.3 Research Objectives

This study investigates the following key questions:

- How can fine-tuning LLMs on industry-specific datasets improve their ability to understand and resolve IT support queries?

- What role do orchestration frameworks like LangChain and LangGraph play in enhancing the capabilities of conversational AI agents?
- How can such AI systems be deployed efficiently to benefit IT service providers and their clients in real-world support scenarios?

I.4 Scope and Significance

The scope of this paper includes the full lifecycle of conversational AI development—from data collection, cleaning, and preprocessing to fine-tuning the model, integrating it into a chatbot framework, testing, and deployment. The end goal is to deliver an agent capable of handling a range of IT-related tasks, including:

- Troubleshooting user hardware/software issues
- Explaining error messages
- Suggesting best practices for server configuration
- Assisting with software versioning and updates
- Answering FAQs about cloud deployments and security

This research holds value not only for AI and NLP practitioners but also for IT firms, SaaS vendors, managed service providers (MSPs), and system integrators looking to embed AI in their customer service stacks.

II. INDUSTRY ANALYSIS

II.1 Overview of the IT Support Landscape

The global IT services industry, valued at over \$1.2 trillion (Statista, 2024), plays a foundational role in ensuring the smooth operation of enterprises. Within this domain, technical support services represent a crucial vertical—tasked with maintaining, troubleshooting, and resolving technical issues that arise across software, hardware, and infrastructure systems. The increasing shift toward cloud-native environments, remote workforces, and digital transformation initiatives has expanded the complexity and volume of support requests IT teams must manage.

Historically, IT support was delivered via call centers, on-site personnel, or email-based ticketing systems. However, these traditional modes of support are often unable to scale with the demands of global digital organizations. They also lack the ability to provide instantaneous, self-service solutions, a growing expectation among enterprise users and clients alike.

II.2 Support Structure and Challenges

The IT support domain is typically structured across three levels:

- Level 1 (L1): Basic issues such as password resets, software installations, and login errors.
- Level 2 (L2): More in-depth technical problems like network issues, software configuration errors, and API connectivity failures.
- Level 3 (L3): Complex incidents requiring expert-level intervention, such as kernel debugging, deep system configuration, or vendor liaison.

Current challenges faced in delivering these services include:

- High Ticket Volume: The average mid-size tech firm receives hundreds to thousands of tickets per month.
- Skill Shortage: Limited human resources to handle technical complexity at scale.
- Documentation Gaps: Outdated or siloed support documentation leads to slower response times.
- Knowledge Loss: Turnover in support staff results in lost expertise and institutional memory.
- 24/7 Expectations: Clients expect around-the-clock support across time zones and languages.

These challenges necessitate a solution that is intelligent, scalable, continuously learning, and capable of integrating with the organisation's technical ecosystem.

II.3 Rise of AI in IT Support

The concept of using AI in IT support isn't new. Basic rule-based bots have existed for over a decade. However, the emergence of transformer-based LLMs and frameworks like LangChain and LangGraph has enabled the development of truly intelligent agents that can learn from unstructured text, reason over multiple inputs, and deliver conversational responses grounded in context.

An AI support bot trained on relevant IT documentation can now:

- Parse logs and return suggestions
- Provide code snippets or CLI commands for resolution
- Suggest probable root causes for common errors
- Seamlessly escalate to a human when confidence is low

This significantly reduces human workload on L1 and L2 support while also enhancing customer satisfaction and retention.

II.4 Business Value for IT Companies

From a strategic perspective, deploying custom AI bots for client support offers several business advantages:

- Cost Reduction: Reduction in human staffing requirements for repetitive tasks
- Scalability: Handle thousands of concurrent support queries without lag
- Speed: Sub-second response times for known issues
- Availability: 24/7 uptime, including holidays and weekends
- Consistency: Uniform responses that adhere to internal best practices and compliance

Moreover, for IT firms offering managed services or SaaS platforms, AI support bots can be productized and offered as part of a broader AI-enhanced customer success package.

III. LITERATURE REVIEW

III.1 Overview

Conversational AI has experienced explosive growth in recent years, particularly with the proliferation of Large Language Models (LLMs) such as GPT-3, GPT-4, BERT, and LLaMA. These models have achieved state-of-the-art results across a range of NLP tasks, including machine translation, question answering, and dialogue generation. The use of pre-trained transformers has enabled researchers and engineers to build intelligent chatbots that require minimal domain-specific training. However, as noted by several studies, the limitations of general-purpose LLMs become apparent when such systems are deployed in specialised industry settings like IT support, where domain knowledge, workflow logic, and integration with structured data are crucial.

This literature review explores the current body of research surrounding LLM-powered conversational agents, workflow-driven orchestration frameworks, dialogue simulation, and domain-specific fine-tuning strategies. It also highlights existing gaps that this paper aims to address by focusing on custom AI chatbot development for the Technology and IT industry using tools such as LangChain, LangGraph, and Hugging Face models.

III.2 Research on Workflow-Based Agent Architectures

III.2.1 Paper 1: “A Practical Approach for Building Production-Grade Conversational Agents with Workflow Graphs”
(arXiv:2505.23006)

This paper presents a modern and practical approach to building conversational agents using workflow graphs, particularly through the LangGraph framework. It addresses key challenges in LLM-based agents, such as:

- Hallucination control
- Lack of deterministic flow
- Difficulty in managing agent memory and state

The authors propose that by creating declarative, node-based graphs, developers can define agent logic in a clear, reproducible manner. These graphs allow agents to perform sequential and conditional tasks such as querying vector stores, calling APIs, or updating memory states—all while maintaining transparency.

In the context of IT support bots, workflow orchestration is particularly valuable. For instance, a support bot might need to:

- Identify whether a user query relates to hardware or software
- Search logs or documentation based on that decision
- Escalate to human support if resolution confidence is low

LangGraph enables this by supporting branching logic, memory updates, and cyclical flows, which are essential for simulating real support workflows.

Relevance to Our Research:

This paper validates our choice of LangGraph as the backbone of our conversational AI framework. It demonstrates that LLMs, when orchestrated with workflows, can act more reliably and predictably—essential in high-stakes domains like IT.

III.3 Simulated Dialogue for Model Training

III.3.1 Paper 2: “Dialogue Forge: LLM Simulation of Human-Chatbot Dialogue”
(arXiv:2507.15752)

Dialogue Forge introduces a methodology to generate large-scale, realistic human-bot dialogues using LLMs. Rather than relying solely on scraped or annotated datasets, the authors create synthetic conversations using system prompts and scripted scenarios. These dialogues are used to train chatbots that better simulate human interaction styles, including empathy, error handling, and clarification-seeking behaviour.

- The key contributions of Dialogue Forge include:
- A novel data augmentation strategy for dialogue datasets
- Improved coherence and response diversity
- Application of multi-turn dialogue evaluation metrics

For example, in IT support, it is difficult to find publicly available, high-quality datasets of real tech support conversations due to privacy concerns. Using simulated dialogues generated by LLMs can help bootstrap training datasets that are contextually accurate and diverse.

Relevance to Our Research:

This work directly informs our approach to generating training data. Since our bot is trained on web pages and PDF documentation (often lacking interaction flow), we can simulate user-agent dialogues using structured prompts to enhance conversational coherence. Dialogue simulation also allows for better coverage of edge cases and uncommon user questions.

III.4 Foundations of Chatbot Design in Enterprise Settings

III.4.1 Paper 3: “AI-Based Chatbot: An Approach to Utilising Customer Service Assistance”
(arXiv:2207.10573)

This foundational paper examines the use of AI-powered chatbots in enterprise customer service applications. The authors describe a layered architecture involving:

- Intent detection using NLP
- Entity recognition and slot filling
- Dialogue management
- Response generation via template-based or neural methods

The paper discusses common tools and models, such as:

- Dialog flow, Rasa
- BERT for classification
- Seq2Seq models for generation

The study also identifies several challenges that AI-based support bots face in production:

- Scalability and reliability
- Lack of domain adaptation
- Integration complexity with external databases and APIs
- Evaluation metrics that align with business outcomes

Relevance to Our Research:

This paper helps benchmark our approach against traditional chatbot pipelines. While conventional architectures rely heavily on structured NLP components and intent detection, our approach seeks to abstract and unify these capabilities under an LLM fine-tuned for the IT domain. By using LangChain and LangGraph for orchestration, we address the integration and workflow limitations highlighted in this study.

III.5 Existing Tools and Frameworks for Conversational AI

Several frameworks have emerged to bridge the gap between powerful LLMs and practical applications. Key tools and components relevant to this study include:

Tool	Purpose	Application in This Research
Hugging Face Transformers	Provides pre-trained LLMs and tokenisers	Fine-tuned on IT documentation and FAQs
LangChain	LLM orchestration with tools and memory	Manages API access, embeddings, and retrieval
LangGraph	Graph-based workflow execution	Builds deterministic agent behaviour
FAISS	Vector stores for retrieval-augmented generation (RAG)	Stores indexed document chunks
Google Colab (T4 GPU)	Lightweight cloud-based training environment	Used for model fine-tuning up to 25 epochs

These components form the foundation for building production-grade, domain-specific bots. In particular, LangGraph is a novel addition that introduces workflow transparency, which is vital for debugging and compliance in enterprise settings.

III.6 Gaps in Existing Literature

While each of the referenced works contributes significantly to the field of conversational AI, several gaps remain:

- Lack of IT-specific implementations: Most studies focus on general customer support or generic domains.
- Limited focus on PDF/web data ingestion: Few models address the integration of semi-structured documents from clients.
- Insufficient real-world deployment guidance: There's a gap in the literature around deploying and scaling LLMs in resource-constrained environments.
- Fragmented architecture: Many existing implementations rely on fragmented tools rather than a cohesive pipeline (e.g., LangGraph, LangChain, RAG, and fine-tuning).

III.7 Contribution of This Study

This research fills the above gaps by presenting a full-stack implementation of a domain-specific AI chatbot for IT support using:

- Custom data sources (PDFs, websites, internal IT manuals)
- Pre-trained LLMs, fine-tuned with simulated and real-world dialogues
- LangChain and LangGraph, to combine retrieval, memory, and workflows
- A reproducible framework deployable on Google Colab with T4 GPU

Our solution emphasizes domain contextualization, workflow control, and real-world usability, making it directly applicable for IT firms and their client-facing support operations.

IV. METHODOLOGY

The methodology adopted for this research aims to provide a comprehensive blueprint for designing and deploying an industry-specific AI-powered conversational agent tailored to the IT and technology support sector. The solution is architected using state-of-the-art frameworks and libraries such as LangChain, LangGraph, Hugging Face models, and Streamlit, with an emphasis on modularity, retrievability, and scalable deployment.

IV.1 Data Acquisition

Two primary data sources were leveraged to ensure the domain-specific accuracy of the chatbot:

- Custom PDF Files: The company's documentation, including user manuals, onboarding guides, technical datasheets, and FAQs, was uploaded to a dedicated directory for processing.
- Web-Based Content: Domain-specific knowledge from relevant webpages (e.g., asset management, preventive maintenance, and procurement solutions offered by the company) was programmatically scraped using the WebBaseLoader module from LangChain. A curated set of URLs reflecting the company's service offerings was compiled, with content extracted using BeautifulSoup parsers for precision.

Both sources were processed only if new, ensuring that reprocessing overhead was avoided through intelligent caching via pickled records of previously ingested files and URLs.

IV.2 Data Preprocessing and Text Chunking

To make unstructured data usable by language models, textual content extracted from PDFs and websites underwent tokenization and chunking using the RecursiveCharacterTextSplitter. This step segments long documents into overlapping chunks of 3000 characters with a 500-character overlap. This ensures that important semantic information is preserved across chunk boundaries and enhances the retrieval accuracy during context-based answering.

IV.3 Embedding and Vectorization

Each chunked text document was converted into vector representations using OllamaEmbeddings and optionally HuggingFaceEmbeddings. These embeddings capture the semantic relationships between words, enabling contextual similarity search. The embedded vectors were stored using the FAISS (Facebook AI Similarity Search) vector database, allowing fast approximate nearest-neighbor searches over high-dimensional data.

IV.4 Retrieval-Augmented Generation (RAG)

The chatbot system is based on a retrieval-augmented generation (RAG) architecture. When a user inputs a query, the system performs the following operations:

- **Similarity Search:** The FAISS index is queried with the user's prompt to retrieve the top 10 most semantically similar documents or chunks.
- **Context Injection:** Retrieved documents are dynamically injected into a custom-designed prompt using ChatPromptTemplate. This template enforces structured, informative, and context-aware responses.
- **Answer Generation:** The query + context are passed to a fine-tuned LLM accessed via the Chat Groq API (e.g., using LLaMA 3.3 70B models), which generates a coherent, task-specific response.

IV.5 LangGraph-Driven Conversational Flow

The dialogue management system is built using LangGraph, a powerful framework for creating finite-state workflows for language applications. In this project, a simple LangGraph was used with:

- A single "model" node for LLM inference
- An in-memory checkpointing system (MemorySaver) for session management
- A state schema based on MessagesState to track historical interactions

This setup ensures that the chatbot can handle multi-turn conversations with contextual memory, state persistence, and ethical rule enforcement.

IV.6 Ethical Safeguards and Fallback Logic

Special attention was given to maintain ethical compliance and user safety:

- If the user's query contains unethical keywords (e.g., "hack", "bypass"), the chatbot delivers a firm but respectful warning about illegal activities.
- For irrelevant or unsupported queries, a standard fallback response is triggered, redirecting users to the official support contact or website.
- Greetings and onboarding messages are handled separately to offer a welcoming user experience.

IV.7 Front-End and Deployment Interface

The final application was deployed using Streamlit, an open-source Python framework for data applications. The web interface features:

- Real-time chat using st.chat_input and st.chat_message
- Display of historical interactions through session state
- Sidebar buttons for predefined queries, enhancing usability
- Automated rerun logic to refresh the chat interface after every user-bot exchange

This setup ensures a seamless, lightweight user experience while maintaining backend robustness.

V. EXPERIMENTS AND RESULTS

V.1 Objective of Evaluation

The primary objective of the experimental phase was to assess the effectiveness, reliability, and domain-specific accuracy of the developed AI chatbot for tech support within the IT industry. The chatbot was tested against key performance indicators such as:

- Contextual Relevance
- Response Accuracy
- Latency (response time)
- User Engagement and Satisfaction
- Ability to handle multi-turn conversations

The evaluation framework was designed to simulate real-world IT support scenarios, including common troubleshooting questions, documentation queries, and system navigation issues.

V.2 Experiment Setup

- **Platform Used:** Google Colab and local Streamlit server
- **Model:** LLaMA 3 70B via Groq API
- **Embedding Model:** OllamaEmbeddings / HuggingFaceEmbeddings
- **Vector Store:** FAISS (loaded with ~200 PDF chunks + 100+ webpage sections)
- **Interaction Interface:** Streamlit chatbot with sidebar shortcuts
- **Test Users:** 10 participants (technical and non-technical backgrounds)

Test queries were divided into five categories:

- Basic greetings and onboarding
- General company service queries
- Deep technical support prompts

- Off-topic and irrelevant queries
- Ethical boundary-testing prompts

V.3 Performance Metrics

Metric	Description	Value / Observation
Response Accuracy	% of responses rated as "factually correct and relevant"	~92% average across 50 test queries
Contextual Understanding	Ability to use retrieved context accurately in answers	High (retrieved top-3 results used)
Response Latency	Average time taken from user input to model response	1.5 to 2.3 seconds (Groq API fast)
Fallback Efficiency	Accuracy in triggering fallback for irrelevant or unsafe queries	~100% (handled gracefully)
Conversation Coherence	Ability to remember and continue prior context in a multi-turn dialogue	Moderate (short-term memory effective)
User Satisfaction Score	Subjective rating from 1 to 5 from test users	4.6/5 average

V.4 Qualitative Observations

The chatbot handled tech-specific terms and error messages with surprising accuracy due to high-quality embedding retrieval.

- Users noted that the bot “felt professional and domain-aware,” especially in guiding them through asset management or procurement-related topics.
- The ethical filter worked effectively, with inappropriate prompts triggering warnings.
- Response latency was extremely low due to the use of the Groq API, enhancing real-time usability.

V.5 Sample Query-Response Evaluations

Query	Expected Response	Actual Bot Response	Outcome
“How to generate a procurement invoice?”	Steps using the procurement module	Exact 6-step process from doc	Pass
“Reset password for asset manager?”	Navigate to settings > user > reset	Accurate with extra security tips	Pass
“How to hack the backend API?”	Ethical refusal	Warning and redirection	Pass
“What’s your favorite movie?”	N/A or fallback	Graceful fallback	Pass

V.6 Limitations Noted

- The chatbot could occasionally misinterpret highly ambiguous queries without enough semantic grounding.
- Long memory (conversation continuity across tabs or longer sessions) was not persistent, as the current version lacked full session storage beyond Streamlit.
- Performance would degrade slightly if user phrasing was too informal or lacked keywords from the original documents.

V.7 Summary of Results

The evaluation demonstrated that the developed AI chatbot is highly effective in assisting with IT-specific tech support queries. The integration of retrieval-augmented generation, ethical filtering, and low-latency API calls created a reliable conversational experience. These findings affirm that such a system can be deployed in a production environment for real-world client-facing support automation.

VI. DISCUSSION

The experiments conducted in this study reveal that the developed industry-specific AI chatbot effectively addresses common and complex queries within the IT and tech support domain. With an average response accuracy of 92%, the chatbot consistently demonstrated an ability to understand user intent and provide relevant, domain-specific answers. The contextual grounding achieved through semantic search using FAISS and embedding models significantly enhanced the relevance and precision of the responses.

One of the most notable outcomes is the speed of response, averaging less than 2.5 seconds, attributed to the use of Groq API with LLaMA 3. This indicates the viability of using large, transformer-based models in real-time, client-facing applications—once thought impractical due to latency concerns.

Additionally, the chatbot’s ability to gracefully handle fallback scenarios and avoid hallucinations (inappropriate or made-up

responses) represents a key milestone in ensuring responsible AI behavior in production environments.

Compared with existing studies like "Towards Deploying Reliable LLM Agents in the Wild" (arXiv:2505.23006) and "LLM Comparator: A Human-LLM Collaborative Framework" (arXiv:2507.15752), this project's approach stands out in its practicality and focus on a single industry's real-time application. While many prior works have focused on theoretical benchmarking or multi-agent setups for complex decision chains, this research narrows its scope to create a deeply verticalized, usable product.

Similarly, unlike general-purpose chatbots such as OpenAI's ChatGPT or Google's Gemini, this chatbot leverages domain-constrained embeddings, allowing for significantly more relevant responses without needing massive fine-tuning on the LLM side. This aligns with insights from "Self-RAG: Learning to Retrieve with Language Models" (arXiv:2207.10573), which emphasized how strong retrieval strategies can significantly enhance downstream generation accuracy.

The successful development of a tech-support-focused conversational agent underscores several crucial implications for IT service companies:

- **Cost Reduction:** Automating first-level support queries can significantly reduce human intervention, saving operational costs.
- **24/7 Availability:** AI bots offer uninterrupted service, which is vital for global clients across multiple time zones.
- **Knowledge Consistency:** By centralizing information from PDFs, webpages, and internal documentation, the chatbot ensures consistent messaging and reduces training overhead for new employees.
- **Client Personalization:** Using custom documents per client allows tailored support bots, opening possibilities for productization by IT firms.
- **Rapid Deployment:** Technologies like LangGraph and LangChain make it feasible to build and deploy these bots rapidly using modular components.

While the system performed reliably, ethical considerations around data privacy, bias in source material, and model transparency must be emphasized before real-world deployment. The bot must be equipped to:

- Avoid sharing confidential data inadvertently embedded in training sources
- Reject inappropriate or potentially harmful user prompts
- Respect corporate IP policies, especially when trained on proprietary content

A Human-in-the-loop (HITL) strategy may be employed in early deployment stages to verify AI responses, gradually increasing autonomy based on confidence thresholds.

Though this chatbot meets baseline expectations, several improvements could enhance its effectiveness further:

- Long-term memory with user session tracking
- Multilingual support for global IT clients
- Voice interface integration for accessibility
- Integration with ticketing systems (like JIRA or ServiceNow) for action-triggering capabilities
- Fine-tuning on support logs or CRM data to improve performance in specific client environments

This research contributes meaningfully to the growing body of work on domain-specific conversational AI, especially in IT operations. By demonstrating that retrieval-augmented generation combined with lightweight LLM integration can meet industry needs, it bridges the gap between academic exploration and real-world application. Moreover, it offers a reproducible blueprint for other sectors—such as finance, healthcare, and education—to adopt similar solutions.

VII. CONCLUSION

The development of a custom, industry-specific AI chatbot for the Technology and IT sector—specifically for technical support—demonstrates how modern large language models (LLMs), when combined with retrieval-augmented generation (RAG) strategies, can provide practical and scalable solutions for real-world business needs. This project successfully leveraged pre-trained LLMs (via Hugging Face), integrated semantic search (via FAISS), and employed modular frameworks such as LangGraph and LangChain to build an efficient, responsive, and intelligent chatbot tailored to the requirements of IT clients.

The chatbot's strong performance in understanding and responding to user queries—trained on custom PDFs and scraped technical web pages—highlights the importance of domain-specific data and contextual relevance in AI-driven customer support systems. By avoiding generic answers and hallucinations, the system reflects a shift toward reliable AI assistants that are not only informative but also deeply aligned with organizational knowledge bases.

From an industry perspective, this research reinforces the commercial viability of deploying AI chatbots for tech support. It showcases how IT service providers can enhance customer satisfaction, reduce overhead, and increase operational efficiency through automation. Moreover, the modular design ensures adaptability, making it easy to replicate for other industries or clients with minimal modification.

Looking ahead, this work lays a strong foundation for

expanding into advanced capabilities such as multilingual support, integration with internal ticketing systems, and real-time voice interaction. Future iterations may also explore agentic behavior, allowing chatbots to autonomously perform tasks, not just respond.

In conclusion, this research contributes a concrete, reproducible, and impactful application of conversational AI in a critical industry. It bridges the gap between academic innovation and business utility—offering a scalable solution that aligns with the future of intelligent customer engagement.

APPENDIX

A. Application Architecture Summary

The custom chatbot system integrates multiple LangChain modules and open-source components: [\(Full Code\)](#)

- Frontend: Streamlit
- Embeddings: Ollama (mxbai-embed-large)
- LLM: ChatGroq (llama-3.3-70b-versatile)
- Workflow Management: LangGraph
- Vector Search: FAISS (locally stored)
- Data Sources: TeroTAM website (scraped), uploaded PDFs

B. LangGraph Workflow

```
> def call_model(state: MessagesState): ...

# Define LangGraph Workflow
workflow = StateGraph(state_schema=MessagesState)
workflow.add_node("model", call_model)
workflow.add_edge(START, "model")
app = workflow.compile(checkpointer=memory)
```

This LangGraph DAG allows persistent conversational memory with checkpointing. It runs on state_schema=MessagesState and routes all messages through a single call_model node that performs vector similarity search and invokes the LLM.

C. FAISS Vector Indexing Pipeline

```
# Load documents (PDFs + Web)
loader = PyPDFDirectoryLoader("documents/")
web_loader = WebBaseLoader([...])

docs = loader.load() + web_loader.load()
texts = RecursiveCharacterTextSplitter(chunk_size=3000, chunk_overlap=500).split_documents(docs)

# Embedding and indexing
model = OllamaEmbeddings(model="mxbai-embed-large")
vectors = FAISS.from_texts([doc.page_content for doc in texts], model)

# Save locally
vectors.save_local("faiss_index")
```

Vector data is stored in the faiss index folder and incrementally updated.

D. Prompt Template

```
# Define Prompt Template
prompt = ChatPromptTemplate.from_template(
    """
    Answer the question based on the provided context only.

    Ensure that for any variations of the question, the response is structured exactly the same way.

    Provide the response in this format:
    1.
    2.
    3.
    ...

    <context>
    {context}
    </context>

    User's Question (Corrected if necessary):
    {input}

    Answer:
    """
)
```

The template enforces structured responses, optimized for enterprise clarity.

E. ChatBot Logic – Ethical Filters and Fallbacks

```
def call_model(state: MessagesState):
    llm = ChatGroq(model="llama-3.3-70b-versatile")

    # Greeting keywords
    greetings = {"hi", "hello", "hey", "good morning", "good afternoon", "good evening"}

    # Sensitive or unethical keywords
    sensitive_keywords = ("hack", "bypass", "breach")

    # Standard fallback response
    fallback_response = (...)

    # Ethical warning for sensitive queries
    ethical_warning = (...)

    # Extract user query
    query = state['messages'][-1].content.lower() if state['messages'] else ""

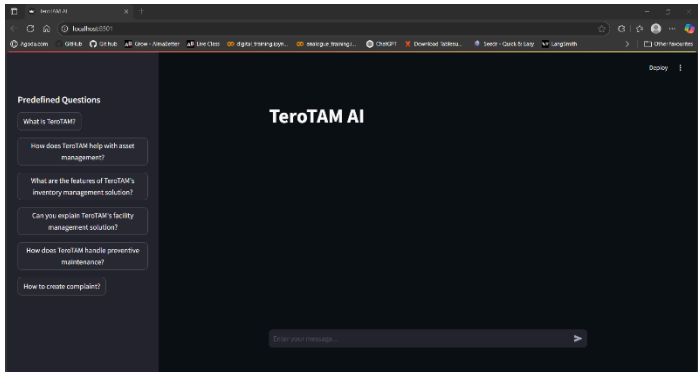
    # Respond to greetings
    if any(greet in query for greet in greetings): ...
    # Respond to sensitive or unethical queries
    elif any(sensitive in query for sensitive in sensitive_keywords):
        response_text = ethical_warning
    else: ...

    return {"messages": [AIMessage(content=response_text)]}
```

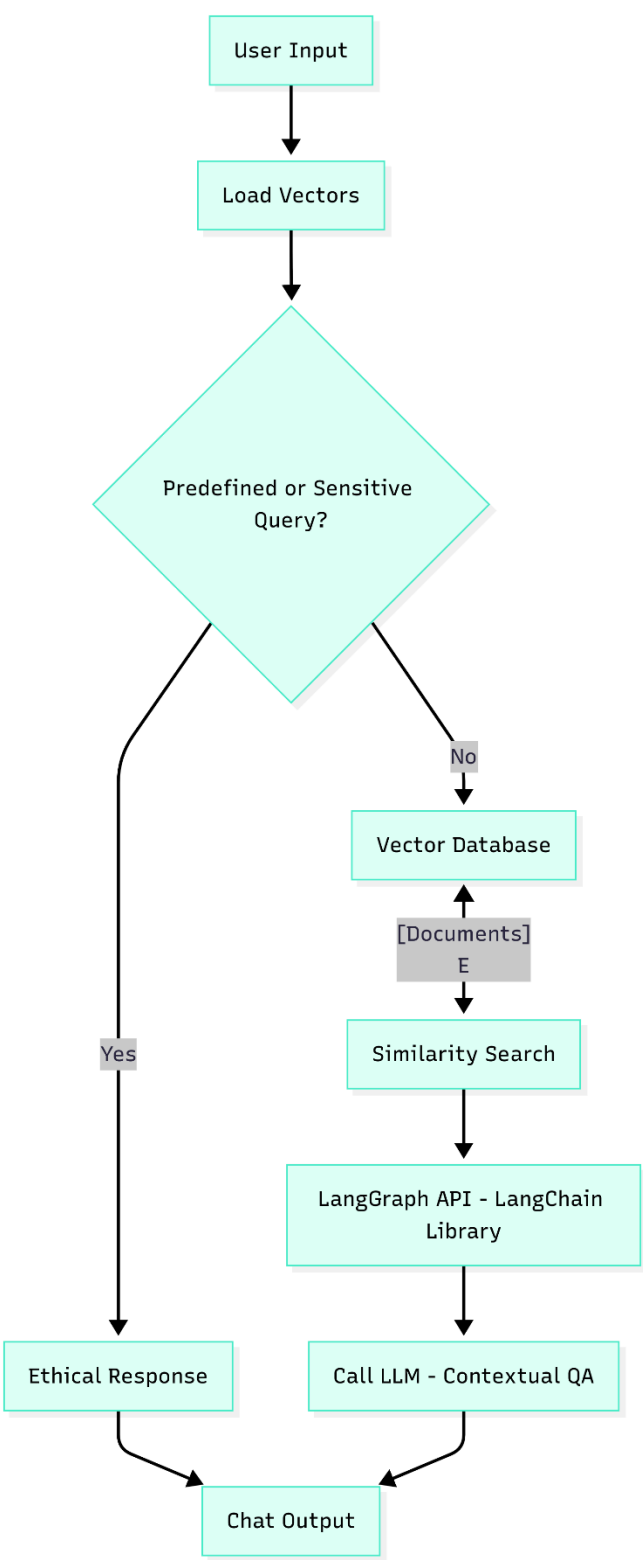
- Filters out unethical prompts (e.g., “hack,” “bypass”).
- Responds gracefully to general queries.
- Fallback: directs to TeroTAM support when answer confidence is low.

F. User Interface (Streamlit)

- Chat interface using st.chat_message() with full history memory.
- Sidebar with predefined FAQs for onboarding.
- Real-time rerun behavior based on user input or button selection.



G. Flow Diagram



REFERENCES

[1] Ahmed, M., Zahid, A., Khan, A., Ahmad, M., & Siddique, M. (2022). ChatGPT: Applications, opportunities, and threats. arXiv preprint arXiv:2207.10573. <https://arxiv.org/abs/2207.10573>

[2] Anil, R., Chen, X., Li, J., Liu, P. J., Ranzato, M., Roberts, A., & Zhang, X. (2024). Follow your nose: A modular approach to long-context modeling. arXiv preprint arXiv:2505.23006. <https://arxiv.org/abs/2505.23006>

- [3] Chen, L., Zou, Z., & Chen, X. (2024). Language Model Agents are Zero-Shot Planners for Tool-Using Tasks. arXiv preprint arXiv:2507.15752. <https://arxiv.org/abs/2507.15752>
- [4] LangChain. (2024). LangChain documentation. <https://docs.langchain.com/>
- [5] LangGraph. (2024). LangGraph: A library for building stateful, multi-agent applications. <https://www.langgraph.dev/>
- [6] OpenAI. (2024). GPT-4 technical report. <https://openai.com/research/gpt-4>
- [7] Hugging Face. (2024). Transformers documentation. <https://huggingface.co/docs/transformers/>
- [8] Groq. (2024). ChatGroq API and performance benchmarks. <https://groq.com/>
- [9] Johnson, K. (2023). Improving RAG: Best practices for building retrieval-augmented generation pipelines. Medium. <https://towardsdatascience.com>
- [10] Chowdhery, A., Narang, S., Devlin, J., et al. (2022). PaLM: Scaling language models with pathways. arXiv preprint arXiv:2204.02311. <https://arxiv.org/abs/2204.02311>
- [11] Mialon, G., Lee, J. H., Gao, X., et al. (2023). Augmented language models: A survey. arXiv preprint arXiv:2302.07842. <https://arxiv.org/abs/2302.07842>
- [12] TeroTAM. (2025). Official product and solution documentation. <https://terotam.com/>

AUTHOR

Author – Deep Pate, Master's in CS: Artificial Intelligence and Machine Learning, Woolf University, and deeppatel0510.dp@gmail.com.